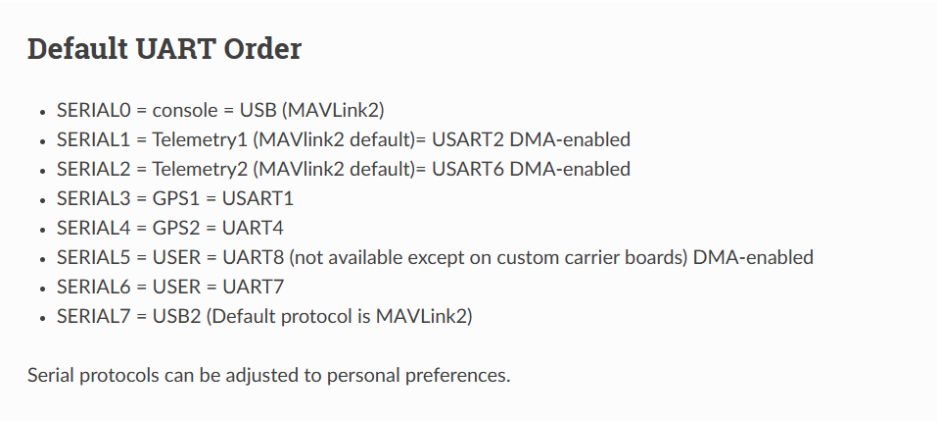
Lidar → Raspberry Pi → CUAV X7+ – Current Status & Findings

1. System summary

- **Lidar** (Unitree L2) → **ROS2 node on Raspberry Pi**
  Node converts point cloud to a 360° `LaserScan` with **72 bins** (0–2π, 5° per bin, 10 Hz).

- **ROS2 → ArduPilot** via **MAVROS ObstacleDistance plugin**, publishing MAVLink `OBSTACLE_DISTANCE` (72 distances) which is the standard interface ArduPilot expects for 360° proximity data.

- Pi is connected to the **CUAV X7+ USB2 (OTG) port**, which ArduPilot exposes as **SERIAL7**. Docs confirm that: SERIAL7 = USB2 (MAVLink2 by default) on X7+.



Default UART Order

- SERIAL0 = console = USB (MAVLink2)
- SERIAL1 = Telemetry1 (MAVlink2 default)= USART2 DMA-enabled
- SERIAL2 = Telemetry2 (MAVlink2 default)= USART6 DMA-enabled
- SERIAL3 = GPS1 = USART1
- SERIAL4 = GPS2 = UART4
- SERIAL5 = USER = UART8 (not available except on custom carrier boards) DMA-enabled
- SERIAL6 = USER = UART7
- SERIAL7 = USB2 (Default protocol is MAVLink2)

Serial protocols can be adjusted to personal preferences.

2. Serial & proximity configuration check

From the parameter set you shared:

- SERIAL7_BAUD = 921 (921600)
- SERIAL7_PROTOCOL = 2 (MAVLink2)

→ This is a correct high-bandwidth link configuration for a companion computer on USB2.
Proximity / avoidance:

- PRX1_TYPE = 2 → "MAVLink" proximity sensor (DISTANCE_SENSOR / OBSTACLE_DISTANCE), which is exactly what MAVROS sends.
- PRX1_MIN = 0.3, PRX1_MAX = 30 → Matches L2 lidar range.
- AVOID_ENABLE = 7 → horizontal avoidance enabled.
- PRX_FILT = 0.25 → very low cutoff for the built-in proximity low-pass filter (strong smoothing, noticeable delay).

Conclusion: wiring and high-level parameter setup on the FC are consistent with ArduPilot docs and with the ROS2/MAVROS pipeline. The large delay you see in the GCS is **most likely coming from (a) the ROS2→MAVROS pipeline or (b) the GCS telemetry link and smoothing**, not from a fundamentally wrong serial mapping.

3. Likely sources of the "big delay"

1. **ROS2 / MAVROS side**

- Even though scan_time ≈ 0.1 s (10 Hz), the node that bins and republishes into MAVROS may be slower (CPU, QoS, queue size).
- If `/scan` is at 10 Hz but the topic that MAVROS subscribes to (e.g. `/mavros/obstacle/send`) is slower, the delay is entirely on the Pi.

2. **Telemetry from FC to GCS**
   - `OBSTACLE_DISTANCE` is sent from Pi → FC at 921600 baud, but from FC → GCS it goes over the radio/USB used by the operator. Those links (often 57 600 baud with many streams enabled) can be congested, so **Mavlink Inspector can show late updates even though the FC already reacted internally.**
3. **Filtering inside ArduPilot**
   - PRX_FILT = 0.25 strongly low-pass filters each proximity sector. ArduPilot docs state that lower values give heavier smoothing and more lag; 0 disables the filter.

4. Recommended parameter tweaks (FC side)

Safe changes to try for reducing perceived latency and improving debuggability:

1. **Proximity filter**
   - For testing, set
- PRX_FILT = 0 (disable filtering)
- or PRX_FILT = 1–2 Hz instead of 0.25, to reduce smoothing delay.
   - Once things are stable, we can re-introduce some filtering if the lidar noise requires it.
2. **Proximity logging**
   - Set PRX_LOG_RAW = 1 so that logs contain both raw and filtered distances; this will help compare:
     - ROS2 timestamps,
     - raw distances at the FC,
     - filtered distances used by avoidance.
3. **(Optional) Telemetry stream tuning**
   - If the delay is only visible on the GCS, we can increase the relevant SRx_* rates for the telemetry port that the pilot uses, or simply accept that the GCS view is slower while the FC is already making decisions.

5. Recommended checks for Nitish on the Pi

To isolate where the real latency comes from:

1. **Topic rates**
- ros2 topic hz <raw_scan_topic> # e.g. /unilidar_lidar/scan
- ros2 topic hz /mavros/obstacle/send # or the remapped LaserScan → MAVROS topic
   - Both should be close to **10 Hz**. If the second one is much lower, the bottleneck is in the ROS2 processing step.
2. **FC receive rate**
   - From a GCS or MAVProxy connected **directly** to the FC, check the frequency of **`OBSTACLE_DISTANCE`** messages (expected ≈ 10 Hz). If it is 10 Hz at the FC, but slower at the GCS radio link, the delay is on telemetry only.

3. **Log-based latency measurement**
    - Compare:
        - LaserScan timestamps on the Pi, and
        - timestamps of proximity updates in the ArduPilot `.bin` logs.
    - This gives the true sensor-to-FC latency, independent of GCS visuals.

Additional reference & wiring notes

While reviewing other ArduPilot + 360° LiDAR integrations, I looked at:

- **Step-by-Step Robotics "Attler" rover** – Jetson Nano + 360° YDLIDAR. The companion converts each scan into **72 angular bins** and sends them as `OBSTACLE_DISTANCE` over MAVLink to the autopilot, exactly like our plan. The Jetson is connected to the FC via **TELEM2 (SERIAL2) at 921600 baud, MAVLink2** using a simple USB-to-UART adapter. - Link [Step By Step Robotics]

- **ArduPilot's RealSense / depth-camera OA example**, which also sends `OBSTACLE_DISTANCE` and recommends checking that Mission Planner sees the message at around **10–15 Hz** in MAVLink Inspector and the *Proximity* view.

From those examples the key configuration ideas that are relevant for our system are:

1. **MAVLink proximity source**
    - Set `PRX1_TYPE = 2` (MAVLink) so ArduPilot treats the Raspberry Pi stream as a proximity sensor.
    - Keep `PRX1_MIN ≈ 0.3 m and PRX1_MAX ≈ 30 m`, matching the LiDAR's published `range_min` / `range_max`.
    - Ensure the Pi publishes **72 distances per message** at **10–15 Hz** (our LiDAR expert already binning to 72, which matches these reference builds).
2. **Obstacle avoidance behaviour**
    - Use `OA_TYPE = 1` (BendyRuler) and `AVOID_ENABLE = 7` so proximity data is used in all auto modes.
    - Set `AVOID_BEHAVE = 1` so the vehicle brakes / stops in front of an obstacle instead of sliding around it by default – this is also what the Attler example uses.
3. **Verification in Mission Planner**

    - On the ground, we should repeat the same checks as in the RealSense guide:

        - Open **MAVLink Inspector** and confirm `OBSTACLE_DISTANCE` messages are arriving at ~10–15 Hz when the LiDAR is running.

- Open **Ctrl-F → Proximity** and verify that the distances change correctly as obstacles are moved around the drone.

USB-C (SERIAL7) vs TELEM ports for the Raspberry Pi

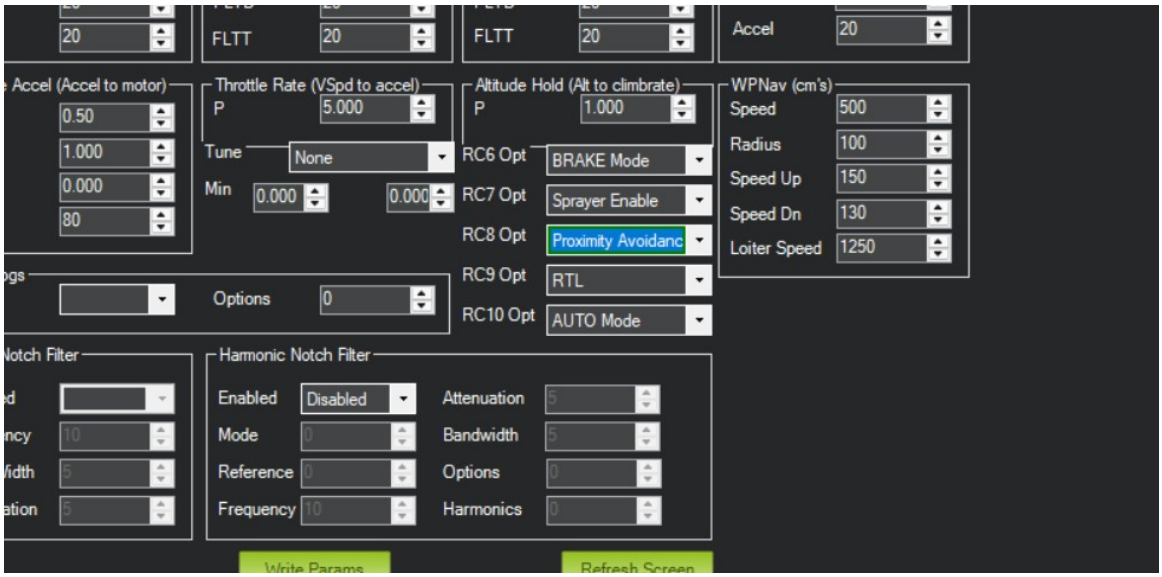
On the CUAV X7+ the default UART mapping is: **SERIAL1 = TELEM1, SERIAL2 = TELEM2 (both DMA-enabled UARTs), SERIAL0 = USB console, SERIAL7 = USB2**, all defaulting to MAVLink2.

- In our current setup, the Raspberry Pi is connected via **USB-C**, which appears as **SERIAL7** and is already configured as MAVLink2 at **921600 baud**. That link is fast enough for 10–15 Hz OBSTACLE_DISTANCE traffic; the delays we see are more likely due to:
  - ROS2 / binning / MavROS processing on the Pi, or
  - timing of the publisher (e.g. publishing slower than 10 Hz), rather than the raw serial bandwidth.
- The Attler reference instead routes the companion through **TELEM2 (SERIAL2)** using a USB-to-UART adapter.
  - Advantage: TELEM1/2 are **DMA-enabled hardware UARTs** dedicated to telemetry, which can give slightly more deterministic timing and keep all companion-computer traffic on a "classic" telemetry port.
  - Trade-off: it needs an extra USB-to-UART adapter and wiring; functionally it is still just MAVLink over a serial link at 921600 baud.

Conclusion for our build

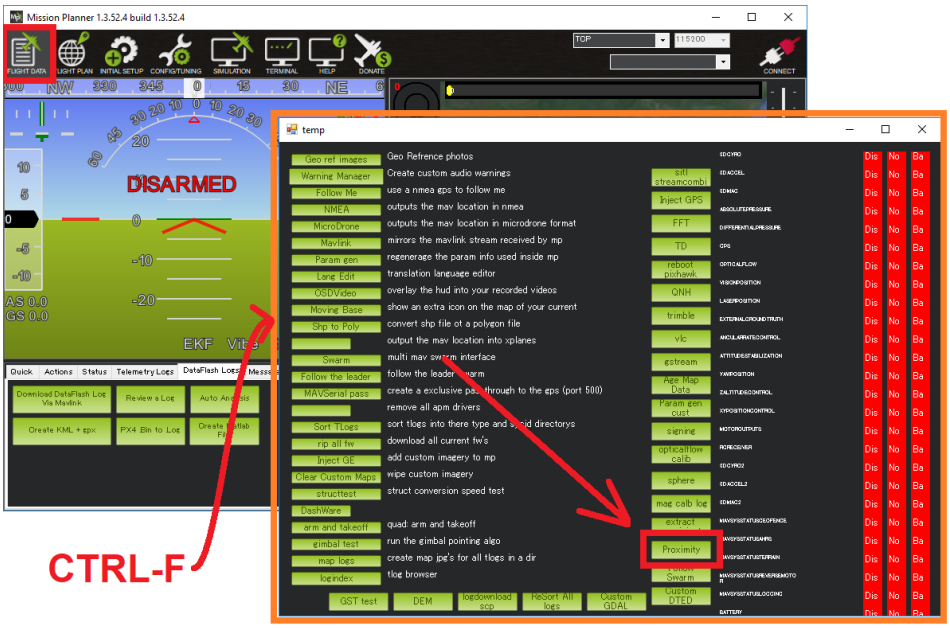
- Staying on **USB-C / SERIAL7** is technically fine for this use case as long as:
  - SERIAL7_PROTOCOL = 2 (MAVLink2) and SERIAL7_BAUD = 921 (921600), which is already true.
  - The Pi publishes OBSTACLE_DISTANCE at **10–15 Hz** and we confirm that rate in Mission Planner's MAVLink Inspector / Proximity view.
- If we continue to see timing jitter or want a completely isolated link for the companion computer, an alternative is to **move the Raspberry Pi to TELEM1 or TELEM2** (SERIAL1 or SERIAL2) like the Attler setup, keeping the configuration otherwise identical.

- **Optional**: You can assign an RC switch to enable Avoidance instead of always on by default. Example of setting RC7 to switch Avoidance on in Mission Planner: m



Ground Test: Pre-Flight Verification

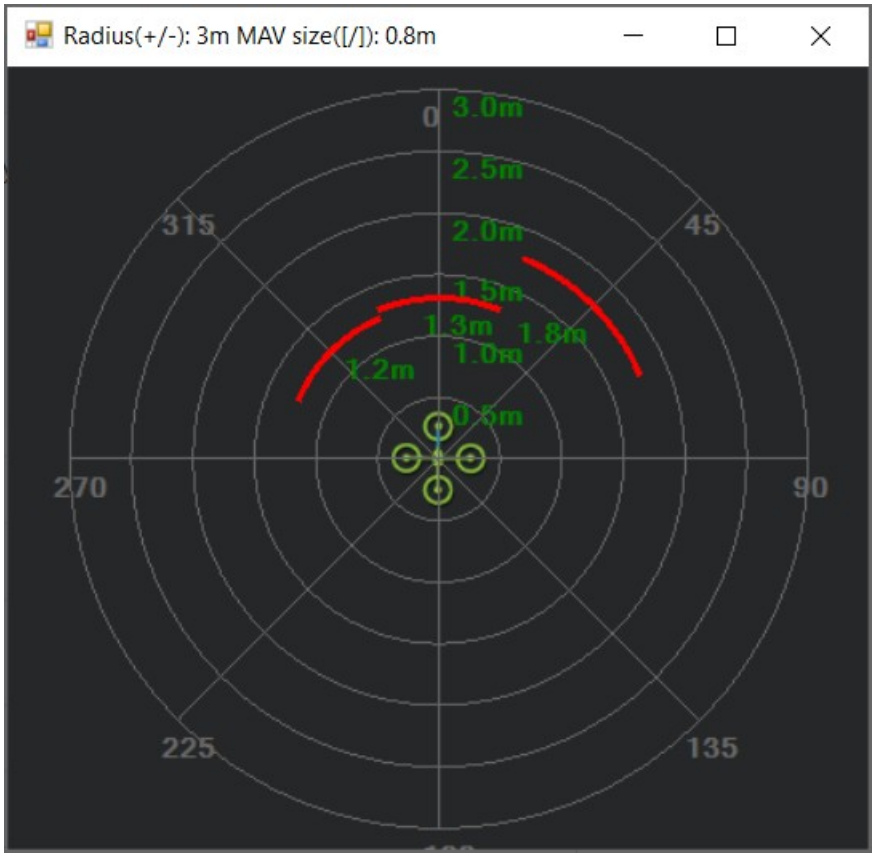
- To verify that the APSync image is working and everything has been correctly configured ensure ArduPilot is receiving OBSTACLE_DISTANCE messages, on Mission Planner: press Ctrl+F and click on “Mavlink Inspector”, you should be able to see data coming in:



If you have a stable telemetry connection, the data frequency for `OBSTACLE_DISTANCE` message should be close to 15 hz. If not, use a USB cable to connect AP and GCS to make sure that the obstacle avoidance data is being received as intended by AP before moving on.

Within Mission Planner, open the `Proximity view` (`Ctrl-F` > `Proximity`):

Put the vehicle/depth camera in front of some obstacles, check that the distance to the nearest obstacle is accurate is shown in the Proximity view.



Stream-rates (Attler example) and USB vs Telemetry

MAV2_ADSB = 10Hz (SR2_ADSB =10Hz)

Command	Value	Units	Options	Desc	Fav
SR2_ADSB	10	Hz	0 50	ADSB stream rate to ground station	
SR2_EXT_STAT	4	Hz	0 10	Extended status stream rate to ground station	
SR2_EXTRA1	4	Hz	0 10	Extra data type 1 stream rate to ground station	
SR2_EXTRA2	4	Hz	0 10	Extra data type 2 stream rate to ground station	
SR2_EXTRA3	4	Hz	0 10	Extra data type 3 stream rate to ground station	
SR2_PARAMS	10	Hz	0 10	Parameter stream rate to ground station	
SR2_POSITION	4	Hz	0 10	Position stream rate to ground station	
SR2_RAW_CTRL	4	Hz	0 10	Raw Control stream rate to ground station	
SR2_RAW_SENS	4	Hz	0 10	Raw sensor stream rate to ground station	
SR2_RC_CHAN	4	Hz	0 10	RC Channel stream rate to ground station	

In the Attler setup the companion computer is connected to a **telemetry UART** (TELEM2) through a USB-serial adapter, while TELEM1 is used for the radio link to the GCS. This keeps the autopilot USB port free and gives a robust fixed connector in flight.

- On our CUAV X7+ the USB-C “USB2” port (SERIAL7) is also a full MAVLink2 port at 921600 baud, so using it for the Raspberry Pi is technically fine. The main trade-offs:

- **Pros:** simple wiring, high bandwidth, already configured as `SERIAL7_BAUD = 921, SERIAL7_PROTOCOL = 2`.

- **Cons:** the USB-C can’t be used for direct PC debugging while the Pi is connected, and unplugging USB in flight is less robust than a soldered telemetry connector.